

HW1 Report (Modeling, Preprocessing, and Comparison)

1) (5%) How I built my model

My pipeline is a unified 4-stage workflow:

1. Preprocess raw JSON (`train.json`, `val.json`, `test.json`) into aligned CSV files with fixed 12-label order.
2. Train a multi-label classifier with sigmoid outputs and macro-F1 based model selection.
3. Tune per-label thresholds on validation logits.
4. Run test inference and export submission in `sample_submission.csv` format.

I implemented two model families:

- Recurrent baseline: **BiLSTM + Multi-Head Self-Attention**.
- Transformer family: pretrained encoder + classification head.

Transformer head options I implemented:

- `linear` head (best in my final runs).
- `label_attention` head (experimental).

Final strong setting (CT-BERT based):

- Backbone: `digitalepidemiologylab/covid-twitter-bert-v2`
- Loss: ASL (`gamma_neg=4.0`, `gamma_pos=1.0`, `clip=0.05`) + positive class weighting
- Regularization: dropout, weight decay, early stopping
- Optimization: AdamW, gradient accumulation, LR scheduler on validation macro-F1
- Post-processing: per-label threshold tuning (instead of fixed 0.5)

This confirms that the complete system is not only the encoder choice; threshold calibration is also a major contributor.

2) (5%) Data preprocessing and imbalance handling

2.1 Preprocessing strategy

I did preprocessing with label consistency as first priority:

1. **Strict label order alignment** from `sample_submission.csv` (12 fixed labels).
2. Text normalization:
 - URL handling (replace with `[URL]` token by default)
 - user mention handling (replace with `[USER]` token by default)
 - emoji handling (`demojize` by default)
 - whitespace normalization
 - sequence truncation to max length 128

3. For RNN mode, optional extra cleaning is available (lowercasing, punctuation removal, stopword removal), but Transformer mode keeps more raw context.
4. Metadata generation (`metadata.json`) includes per-label positive rate and recommended `pos_weight`.

2.2 Class imbalance diagnosis

The dataset is highly imbalanced. In training split (`n=6956`):

- `side-effect`: 38.28% positive (major class)
- `country`: 2.01% positive (`pos_weight` $\approx$ 48.69)
- `religious`: 0.65% positive (`pos_weight` $\approx$ 153.58)

So macro-F1 is hard because minority classes are easy to miss.

2.3 What I did to improve macro-F1 on minority labels

I used a combination of three methods:

1. **Class-aware loss weighting:**
 - For BCE/ASL, use per-label `pos_weight` = `negative/positive` from metadata.
2. **Asymmetric Loss (ASL):**
 - Helps suppress overwhelming easy negatives and focuses on difficult positives.
3. **Per-label threshold tuning:**
 - Search label-specific thresholds on validation set, not one global threshold.

Evidence (best CT-BERT run, default -> tuned):

- Macro-F1: `0.6624` -> `0.7209` (+**0.0584**)
- Minority-label F1 gains:
 - `religious`: `0.5882` -> `0.8333`
 - `conspiracy`: `0.5203` -> `0.6053`
 - `country`: `0.6538` -> `0.6977`

This directly improves the macro average because rare labels are no longer under-predicted by a fixed threshold of 0.5.

3) When implementing the model

3.1 (5%) Did I try pretrained word embeddings (GloVe/Word2Vec)? Influence?

Short answer:

- Yes, I ran a controlled **GloVe** ablation on BiLSTM.
- I used `glove-twitter-25` vectors (via gensim) and initialized BiLSTM embeddings with `--glove-path pretrained/glove_twitter25_vocab.txt`.
- I did not run a separate external Word2Vec file in this round; this section reports measured GloVe influence.

Experimental setup (same hyperparameters, same seed = 42):

- Random baseline: `artifacts/bilstm_ablation_random25_seed42`
- GloVe init: `artifacts/bilstm_ablation_glove25_seed42`
- Embedding dim = 25 for both runs (to match GloVe vectors)
- GloVe vocab coverage: $4905 / 10758 = 45.59\%$

Measured influence:

- **Default macro-F1:** 0.4143 -> 0.4145 (+0.0002, almost no change)
- **Tuned macro-F1:** 0.4337 -> 0.4633 (+0.0295)
- **Default micro-F1:** 0.5233 -> 0.5417 (+0.0184)
- **Tuned micro-F1:** 0.5304 -> 0.5673 (+0.0368)

Interpretation:

- Pretrained static embeddings gave a **moderate positive effect** on BiLSTM after threshold tuning, but little effect at fixed threshold 0.5.
- So the answer is: **yes, I tried pretrained word embeddings (GloVe), and they improved final tuned performance, but the gain is still much smaller than contextual pretrained Transformers.**

3.2 (5%) Did I try attention? Influence? What did the model focus on?

Yes, I tried attention in two forms:

1. **BiLSTM + Multi-Head Self-Attention** (recurrent branch).
2. **Label-specific attention head** on top of Transformer (`head_type=label_attention`).

Influence on performance:

- `ctbert_round3_label_attn` default macro-F1: **0.1791** (very poor)
- Comparable linear-head checkpoint (`ctbert_round4_linear_fine`) default macro-F1: **0.5968**
- Best linear CT-BERT run default macro-F1: **0.6624**

So in my current setup, the label-attention head underperformed strongly versus linear head.

Possible reasons:

- The attention head was more sensitive to noisy tokens (`[USER]`, punctuation-heavy tweets).
- Hyperparameter coupling (pooling mode, ASL settings, threshold behavior) likely made optimization unstable.
- Label-attention has more moving parts than a linear head and may require longer/cleaner tuning.

Attention case studies (from label-attention checkpoint)

Case A: target `religious`

- Tweet contains: "Catholic Church ... urging Catholics to avoid taking Johnson & Johnson's COVID-19 vaccine ..."
- Top focused tokens: `catholics, catholic, vaccine, statement, questions, produced`
- Interpretation: the model attended to religion identity words and vaccine-risk context.

Case B: target `country`

- Tweet contains geopolitical framing: “global diplomacy ... chinese vaccine ... asian countries ... 9-dash line”
- Top focused tokens: **chinese, global, vaccine, accepted, free**
- Interpretation: attention captured nation/geopolitical cues.

Case C: target **side-effect**

- Tweet contains claims like “implanted nano chips ... changes your dna ... patent on you”
- Top focused tokens: **dna, chips, patent, minds, soul**
- Interpretation: attention focused on biomedical/conspiracy-effect terms often associated with side-effect narratives.

Even with meaningful attention patterns, this checkpoint still had weak macro-F1, so interpretability alone did not guarantee better accuracy.

4) (10%) RNN/LSTM/GRU vs Transformer performance comparison

4.1 Quantitative comparison

From my saved runs:

- Recurrent baseline (**formal_bilstm**):
 - default macro-F1: **0.4407**
 - tuned macro-F1: **0.4591**
- Transformer best (**ctbert_final_no_overfit_v3**):
 - default macro-F1: **0.6624**
 - tuned macro-F1: **0.7209**

Gap:

- Default macro-F1 improvement: **+0.2217**
- Tuned macro-F1 improvement: **+0.2618**

So Transformer-based approach clearly performs better in my experiments.

4.2 RNN/LSTM/GRU note

In my final code path and saved artifacts, the recurrent model I actually trained and reported is BiLSTM-based. I did not keep a complete saved GRU checkpoint summary in this repository, so my recurrent-side comparison is based on LSTM-family evidence.

4.3 Why Transformer performed better

Most likely reasons:

1. **Contextual pretraining advantage:**
 - CT-BERT/BERTweet/Roberta already encode rich language semantics before task fine-tuning.
2. **Better handling of noisy social-media text:**
 - Subword tokenization is robust to slang, misspelling, hashtags, and rare tokens.
3. **Long-range dependency modeling:**

- Self-attention captures long-distance relations better than shallow recurrent baselines.

4. **Imbalance-aware training compatibility:**

- ASL + class weighting + threshold tuning worked very effectively with Transformer probabilities.

5. **Data scale vs model capacity match:**

- For this dataset size and label complexity, pretrained Transformer gives stronger feature quality than training recurrent features mostly from scratch.

Final conclusion:

- In this project, **Transformer-based models are the better choice** for macro-F1, especially under severe class imbalance.
- Recurrent models are useful as baselines, but they were not competitive with tuned pretrained Transformers in my results.